Муниципальный этап ВсОШ по информатике (проект)

Методические материалы жюри (7-11 классы)

Учебный год: **2026/27**
Профиль: **программирование**

---

1. Соответствие методическим рекомендациям

1.1. Формальные параметры

- Отдельный вариант для 7-8 классов: **5 задач** (рекомендуемый диапазон 4-6).
- Отдельный вариант для 9-11 классов: **5 задач** (рекомендуемый диапазон 4-6).
- Нарастание сложности от задачи A к задаче E в каждом варианте.
- Автоматическая проверка (краткий ответ и программные задачи).

1.2. Баланс сложности

- 7-8: 2 задачи краткого ответа + 3 программные.
- 9-11: 5 программных, из них 1-2 задачи повышенной алгоритмической сложности (B, E), остальные решаемы классическими техниками.

1.3. Неповторяемость

Задачи являются авторской переработкой и не копируют напрямую примеры из приложений к рекомендациям.

---

2. Вариант 7-8 классы: разбор и оценивание

2.1. Таблица оценивания

№	Задача	Формат	Максимум
A	Шторы-мини	Краткий ответ	100
B	Лифт-инвертор	Краткий ответ	100
C	Маятниковая строка	Программирование	100
D	Шторы-online	Программирование	100
E	Дублирующий редактор	Программирование	100

2.2. Ключи к задачам краткого ответа

A. Шторы-мини

Формула минимума операций:

`h1 + sum(max(0, hi - h(i-1)))`

Для массива `3 3 1 4 4 0 2 2 5 1`:

`3 + 0 + 0 + 3 + 0 + 0 + 2 + 0 + 3 + 0 = 11`

Правильный ответ: `11`
Оценивание: 100/0.

B. Лифт-инвертор

Симуляция команд `UUUSDDDDSSUUUD` при старте с этажа 3 и с учетом инверсии дает:

- конечный этаж: `5`;

- число ударов о границу: `3`.

Правильный ответ: `5 3`
Оценивание: 100/0.

2.3. Разбор программных задач

C. Маятниковая строка

Идея: использовать `deque`.

- нечётная позиция -> `push_back`;
- чётная позиция -> `push_front`.

Сложность: `O(n)` время, `O(n)` память.

Рекомендуемые подзадачи:

- `n <= 5000` -> 30 баллов;
- полные ограничения -> 70 баллов.

D. Шторы-online

Идея: та же формула, что и в задаче A:

`ans = h1 + sum(max(0, hi - h(i-1)))`

Сложность: `O(n)` время, `O(1)` дополнительная память.

Рекомендуемые подзадачи:

- `n <= 5000` -> 30 баллов;
- `hi <= 10^4` -> 20 баллов;
- полные ограничения -> 50 баллов.

E. Дублирующий редактор

Идея: динамика по длине префикса.

Пусть `dp[i]` — минимальное число операций для получения первых `i` символов.

Переходы:

1. `ADD`: `dp[i+1] = min(dp[i+1], dp[i] + 1)`
2. `DUP`: если `2*i <= n` и `s[1..i] == s[i+1..2i]`, то
   `dp[2*i] = min(dp[2*i], dp[i] + 1)`

Проверка равенства блоков может быть сделана через `Z`-функцию, чтобы общий алгоритм был линейным.

Сложность: `O(n)` время, `O(n)` память.

Рекомендуемые подзадачи:

- `n <= 5000` -> 25 баллов;
- `n <= 50000` -> 25 баллов;
- полные ограничения -> 50 баллов.

---

3. Вариант 9-11 классы: разбор и оценивание

3.1. Таблица оценивания

№	Задача	Максимум
A	Стабилизация таймлайна	100
B	Маршрут с двумя режимами	100
C	Цепная склейка отчетов	100
D	Квадраты датчиков	100
E	Сегментация шума	100

3.2. Разбор задач

A. Стабилизация таймлайна

Идея: бинарный поиск по `D` + проверка выполнимости за `O(n)`.

Проверка `ok(D)`:

- поддерживаем `prev` (последнее выбранное значение);
- для `ti` доступен интервал `[ti-D, ti+D]`;
- берем минимально возможное `ai = max(prev+1, ti-D)`;
- если `ai > ti+D`, то данный `D` не подходит.

Сложность: `O(n log V)`, где `V` — диапазон ответа.

B. Маршрут с двумя режимами

Идея: алгоритм Дейкстры на графе состояний `2*n`:

- `(v,0)` — режим day;
- `(v,1)` — режим night.

Переходы:

- по ребру в том же режиме: стоимость `c_day` или `c_night`;
- переключение режима в той же вершине: `+p[v]`.

Сложность: `O((n+m) log n)`.

C. Цепная склейка отчетов

Итоговая длина:

`sum(|si|) - sum(overlap(si, s(i+1)))`

где `overlap(A,B)` — максимальная длина суффикса `A`, равного префиксу `B`.

Вычисление `overlap` рекомендуется делать префикс-функцией (КМП) или двойным хешированием.

Сложность: `O(суммарной длины строк)`.

D. Квадраты датчиков

Идея: использовать `hash set` точек.

Перебирать пары точек на одной горизонтали `(x1,y)` и `(x2,y)` (`x1 < x2`), сторона квадрата `d = x2-x1`, далее проверять наличие верхних вершин `(x1,y+d)` и `(x2,y+d)`.

Чтобы не было двойного счёта, учитывать только квадраты "вверх" (или только "вниз").

Сложность: `O(n^2)`.

E. Сегментация шума

Идея: динамическое программирование.

1. Предвычислить `cost[l][r] = max(a[l..r]) - min(a[l..r])` за `O(n^2)`.
2. Переход:

`dp[j][i] = min(dp[j-1][t] + cost[t+1][i])`, где `t < i`.

`dp[j][i]` — минимум для первых `i` элементов при `j` сегментах.

Сложность: `O(k*n^2)`.

---

4. Эталонные решения (конспект)

Для сборника рекомендуется приложить по задаче:

- минимум 1 эталон на C++17;
- минимум 1 эталон на Python 3 (где допустимо по лимитам);
- 1-2 некорректных типовых решения (для разбора ошибок).

---

5. Тест-план

5.1. Обязательные группы тестов для каждой задачи

1. Минимальные размеры входа.
2. Граничные значения параметров.
3. Стресс-тесты на максимальных ограничениях.
4. Конструкции, ломающие популярные ошибочные подходы.

5.2. Частные акценты по задачам

- C (7-8): длинные строки, чередование и повторяющиеся блоки.
- E (7-8): строки с "почти удваиваемой" структурой.
- B (9-11): граф без пути 1->n; большие стоимости.
- C (9-11): полное перекрытие и нулевое перекрытие между соседними строками.
- D (9-11): много точек на одной горизонтали.
- E (9-11): случаи `k=1`, `k=n`, все элементы равны, строго возрастающий массив.

---

6. Рекомендации по техническим настройкам

1. Проверить одинаковость лимитов и компиляторов во всех аудиториях.
2. Для Python и C++ установить разные, но справедливые лимиты времени.
3. Показать участникам формат ошибок системы (WA/TLE/MLE/RE/CE) в памятке.

---

7. Примечание для оргкомитета

Документы этого комплекта содержат:

- пакет условий для участников (`municipal_stage_2026_27_participants_7_11.md`);
- пакет материалов жюри (данный файл).

При необходимости материалы могут быть напрямую перенесены в Word/PDF с сохранением структуры разделов.